

Bộ Tăng Tốc 2D FFT Cho Xử Lý Ảnh RGB Trên FPGA Zynq-7000

Giang Vinh Huy, Nguyen Quang Nhon, Pham Dinh Phuoc, Nguyen Lam Minh Nhat, Doan Nhat Hoa

Department of Computer Engineering

Faculty of Electrical and Electronics Engineering

Ho Chi Minh City University of Technology and Education (HCMUTE)

Ho Chi Minh City, Vietnam

Abstract—Bài báo này trình bày bộ tăng tốc biến đổi Fourier nhanh hai chiều (2D FFT) cho xử lý ảnh RGB trên nền tảng Zynq-7000/PYNQ-Z1. Hệ thống áp dụng mô hình đồng thiết kế phần cứng/phần mềm, trong đó Processing System chuẩn bị dữ liệu, cấu hình và kiểm tra kết quả, còn Programmable Logic thực hiện khối tính toán FFT. Bộ tăng tốc *fft2d_fixed_top* được mô tả bằng C++ trong Vitis HLS, sử dụng thuật toán radix-2, bảng tra hệ số twiddle và biểu diễn số cố định cho ảnh RGB kích thước 64×64 . IP được tích hợp vào Vivado block design với AXI4-Lite cho điều khiển và AXI master cho truy cập DDR. Thiết kế được kiểm chứng bằng dữ liệu tham chiếu Python/NumPy cùng các báo cáo synthesis, implementation và timing.

Index Terms—2D FFT, tăng tốc FPGA, Zynq-7000, Vitis HLS, xử lý ảnh RGB, đồng thiết kế phần cứng/phần mềm.

I. GIỚI THIỆU

Biến đổi Fourier hai chiều là một công cụ quan trọng trong xử lý ảnh số vì cho phép phân tích ảnh trong miền tần số, nơi các đặc trưng như biên, chi tiết nhỏ, nhiễu và kết cấu được thể hiện rõ hơn. Tuy nhiên, phép biến đổi Fourier rời rạc hai chiều (2D DFT) có chi phí tính toán lớn. Thuật toán biến đổi Fourier nhanh (FFT) giảm đáng kể số phép toán bằng cách phân rã phép biến đổi thành các bước nhỏ có cấu trúc lặp lại [1]. Đối với ảnh hai chiều, 2D FFT thường được thực hiện theo phương pháp hàng-cột, tức là FFT một chiều được áp dụng lần lượt trên các hàng và các cột.

Trong các hệ thống nhúng, việc xử lý 2D FFT hoàn toàn bằng CPU có thể gây ra độ trễ cao, đặc biệt với ảnh RGB gồm nhiều kênh dữ liệu. PYNQ-Z1 sử dụng SoC Zynq-7000, kết hợp Processing System (PS) dựa trên ARM và Programmable Logic (PL) trên cùng một chip [2]. Kiến trúc này phù hợp cho đồng thiết kế phần cứng/phần mềm: PS đảm nhiệm chuẩn bị dữ liệu, cấu hình và kiểm tra kết quả, trong khi PL thực hiện phần tính toán FFT có khả năng song song hóa cao.

Dự án này xây dựng bộ tăng tốc 2D FFT cho ảnh RGB kích thước 64×64 trên PYNQ-Z1. IP *fft2d_fixed_top* được tạo bằng Vitis HLS, tích hợp vào Vivado block design và giao tiếp với PS thông qua AXI4-Lite cho điều khiển và AXI master cho truy cập DDR. Hệ thống tách ảnh thành ba kênh R, G, B, thực hiện 2D FFT riêng cho từng kênh và xuất kết quả phân thực/phần ảo để so sánh với dữ liệu tham chiếu từ Python/NumPy.

Các đóng góp chính của dự án gồm:

- Hiện thực bộ tăng tốc 2D FFT cho ảnh RGB 64×64 bằng Vitis HLS theo phương pháp hàng-cột.
- Tích hợp IP *fft2d_fixed_top* vào hệ thống Zynq-7000 với AXI4-Lite điều khiển và AXI master truy cập DDR.
- Kiểm chứng thiết kế bằng Python/NumPy và các báo cáo synthesis, implementation, timing.

Phần còn lại của bài báo được tổ chức như sau. Mục II trình bày cơ sở lý thuyết, Mục III mô tả kiến trúc hệ thống, Mục IV trình bày quá trình hiện thực, Mục VI thảo luận kết quả đánh giá, và Mục VIII tổng kết các hướng phát triển tiếp theo.

II. CƠ SỞ LÝ THUYẾT VÀ CÔNG TRÌNH LIÊN QUAN

A. Biến Đổi Fourier Nhanh Hai Chiều

Với một ảnh $f(x, y)$ có kích thước $M \times N$, biến đổi Fourier rời rạc hai chiều (2D DFT) được định nghĩa như sau:

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) e^{-j2\pi(\frac{ux}{M} + \frac{vy}{N})}. \quad (1)$$

2D DFT chuyển ảnh từ miền không gian sang miền tần số, trong đó các hệ số tần số thấp biểu diễn thành phần biến thiên chậm, còn các hệ số tần số cao thể hiện biên, kết cấu và chi tiết nhỏ. Do đó, miền Fourier thường được sử dụng trong lọc ảnh, tăng cường ảnh và phân tích đặc trưng ảnh số [3]. Trong đề tài này, ảnh RGB được tách thành ba kênh màu và 2D FFT được áp dụng độc lập trên từng kênh để giữ lại thông tin tần số của từng mặt phẳng màu.

Tính trực tiếp (1) đòi hỏi chi phí tính toán lớn vì mỗi hệ số đầu ra phụ thuộc vào toàn bộ mẫu đầu vào. FFT giảm đáng kể độ phức tạp bằng cách phân rã DFT thành các phép toán butterfly nhỏ hơn và khai thác tính đối xứng của nhân Fourier [4], [5]. Với dữ liệu hai chiều, phép biến đổi có thể được thực hiện theo phương pháp hàng-cột: tính 1D FFT cho từng hàng, lưu ma trận trung gian, sau đó tính 1D FFT cho từng cột. Cách tổ chức này phù hợp với thiết kế phần cứng vì dùng lặp lại cùng một nhân FFT 64 điểm, có luồng truy cập bộ nhớ rõ ràng và thuận tiện để so sánh kết quả với NumPy `fft2`.

B. Tăng Tốc FFT Trên FPGA

FFT phù hợp với FPGA vì thuật toán có cấu trúc đều, gồm các tầng butterfly lặp lại, phép nhân với hệ số twiddle và mẫu

truy cập dữ liệu có quy luật. Các đặc điểm này cho phép hiện thực đường dữ liệu chuyên biệt, dùng bộ nhớ trên chip cho dữ liệu trung gian và khai thác song song hóa tốt hơn so với bộ xử lý đa dụng [6]. Tuy nhiên, thiết kế FFT trên FPGA luôn phải cân bằng giữa thông lượng, độ trễ, tài nguyên phần cứng và độ phức tạp điều khiển.

Các kiến trúc FFT có thể ưu tiên pipeline và streaming để tăng thông lượng, hoặc tái sử dụng khối butterfly để giảm tài nguyên. Tài liệu FFT IP của AMD cũng nhấn mạnh các lựa chọn kiến trúc, scaling và xử lý tràn số khi hiện thực FFT fixed-point trên FPGA [7]. Một số nghiên cứu trước đó đã khai thác tái sử dụng tài nguyên và tổ chức bộ nhớ cho các bộ xử lý FFT 1D/2D trên FPGA [8], [9]. Trong bài báo này, hướng tiếp cận được chọn là một bộ tăng tốc HLS để kiểm chứng, dùng lại nhân radix-2 FFT 64 điểm cho các hàng, cột và ba kênh RGB trên nền tảng Zynq-7000.

C. Số Cố Định Trong HLS

Số học dấu chấm động thuận tiện khi mô phỏng phần mềm, nhưng thường tiêu tốn nhiều tài nguyên khi tổng hợp lên FPGA. Vì vậy, các thiết kế DSP trên FPGA thường dùng số học dấu chấm cố định để giảm LUT, FF, DSP và giúp dễ đạt timing hơn. Đổi lại, người thiết kế phải lựa chọn hợp lý số bit phần nguyên, số bit phần thập phân, chế độ làm tròn và cơ chế xử lý tràn số.

Vitis HLS cung cấp kiểu `ap_fixed`, cho phép mô tả số fixed-point với độ rộng tùy chọn ngay trong C++ [10]. Trong FFT, lựa chọn này rất quan trọng vì giá trị trung gian và hệ số DC có thể tăng lớn sau các tầng butterfly; do đó tài liệu FFT IP của AMD xem độ rộng từ, scaling và tràn số là các yếu tố thiết kế cần quan tâm [7]. Trong đề tài này, định dạng `ap_fixed<28, 20>` được chọn thông qua quá trình quét độ rộng bit và so sánh với kết quả NumPy. Cách chọn này nhằm cân bằng giữa độ chính xác của hệ số FFT và tài nguyên phần cứng trên Zynq-7000.

D. Đồng Thiết Kế Trên Zynq

Zynq-7000 là nền tảng SoC tích hợp Processing System (PS) dựa trên ARM và Programmable Logic (PL) dựa trên FPGA trong cùng một chip [11]. Kiến trúc này phù hợp với các bài toán tăng tốc nhúng, trong đó PS xử lý các tác vụ điều khiển như chuẩn bị dữ liệu, cấu hình tham số và kiểm tra kết quả, còn PL đảm nhiệm các khối tính toán lặp lại và có khả năng song song hóa cao [12].

Trong đề tài này, bộ tăng tốc 2D FFT được xây dựng như một IP phần cứng được điều khiển bởi phần mềm. PS lưu ảnh RGB trong DDR, cấu hình IP qua AXI4-Lite và cung cấp địa chỉ bộ nhớ đầu vào/đầu ra. PL đọc dữ liệu qua AXI master, thực hiện 2D FFT cho từng kênh màu và ghi kết quả phần thực, phần ảo trở lại DDR. Cách tiếp cận memory-mapped này đơn giản, dễ kiểm chứng và phù hợp với giai đoạn xây dựng phiên bản nền tảng bằng Vitis HLS [10].

III. KIẾN TRÚC HỆ THỐNG ĐỀ XUẤT

A. Tổng Quan Hệ Thống

Hệ thống đề xuất được tổ chức theo cấu trúc PS/PL của Zynq-7000. PS đảm nhiệm điều khiển ứng dụng, quản lý bộ

đệm DDR, bảo trì cache, cấu hình bộ tăng tốc và kiểm tra kết quả. PL chứa IP 2D FFT được sinh từ Vitis HLS. Bộ tăng tốc đọc ảnh RGB đầu vào từ DDR, tính FFT cho từng kênh màu và ghi các ma trận phần thực/phần ảo về DDR. Kiến trúc memory-mapped được chọn cho phiên bản baseline vì dễ kiểm chứng và phù hợp trực tiếp với các giao tiếp AXI4-Lite và AXI master do Vitis HLS sinh ra [10], [13].

Hình 1 nhấn mạnh sự phân chia giữa điều khiển và tính toán. AXI4-Lite được dùng cho các thanh ghi điều khiển bằng thông thấp, còn AXI master memory-mapped được dùng cho đọc/ghi dữ liệu đầu vào và đầu ra. PS chuẩn bị bộ đệm RGB, cấu hình IP HLS, khởi động bộ tăng tốc và kiểm tra kết quả. PL thực hiện các phép FFT hàng-cột lặp lại.

B. Luồng Xử Lý Ảnh RGB

Ảnh đầu vào được biểu diễn thành ba ma trận 64×64 tương ứng với các kênh đỏ, xanh lá và xanh dương. Mỗi kênh được xem như một ảnh vô hướng độc lập. Bộ tăng tốc tính 2D FFT cho từng kênh và tạo hai nhóm đầu ra cho mỗi kênh: hệ số phần thực và hệ số phần ảo.

Phổ log-magnitude hữu ích để quan sát trực quan cấu trúc miền tần số, nhưng không được dùng làm tiêu chí đúng sai chính. Kiểm chứng định lượng được thực hiện bằng cách so sánh hệ số thực và ảo thô với kết quả tham chiếu Python NumPy.

C. Phương Pháp FFT Hàng-Cột

2D FFT được hiện thực bằng phân rã hàng-cột. Với mỗi kênh màu, bộ tăng tốc trước hết thực hiện FFT một chiều trên từng hàng. Kết quả trung gian được lưu trong một ma trận phức nội bộ. Sau đó, bộ tăng tốc thực hiện FFT một chiều trên từng cột của ma trận trung gian và ghi hệ số phần thực/phần ảo cuối cùng vào các bộ đệm đầu ra. Cách phân rã này phù hợp với tính tách biến của 2D DFT và dễ ánh xạ sang C/C++ HLS.

IV. HIỆN THỰC PHẦN CỨNG/PHẦN MỀM

A. Kiến Trúc Bộ Tăng Tốc HLS

Hàm top của HLS được định nghĩa như sau:

```
void fft2d_fixed_top(  
    data_t input[3][64][64],  
    data_t output_real[3][64][64],  
    data_t output_imag[3][64][64]);
```

Khi biên dịch tham chiếu trên desktop, `data_t` có thể được định nghĩa là `double`. Khi tổng hợp bằng Vitis HLS, thiết kế sử dụng `ap_fixed<28, 20, AP_RND, AP_SAT>` làm kiểu số cố định được chọn. Tổng độ rộng 28 bit cung cấp độ chính xác phần thập phân, trong khi 20 bit phần nguyên cung cấp dải biểu diễn đủ lớn cho các giá trị trung gian quan sát được trong các phép thử FFT 64×64 . Thiết kế dùng kích thước ảnh và số kênh tĩnh, nên cấu trúc vòng lặp và footprint bộ nhớ có tính xác định.

Hình 3 trình bày các khối chính của datapath. Điểm hiện thực quan trọng nhất là thay thế tính toán lượng giác runtime bằng bảng tra twiddle 64 điểm. Cách này tránh việc HLS tổng

Zynq-7000 SoC

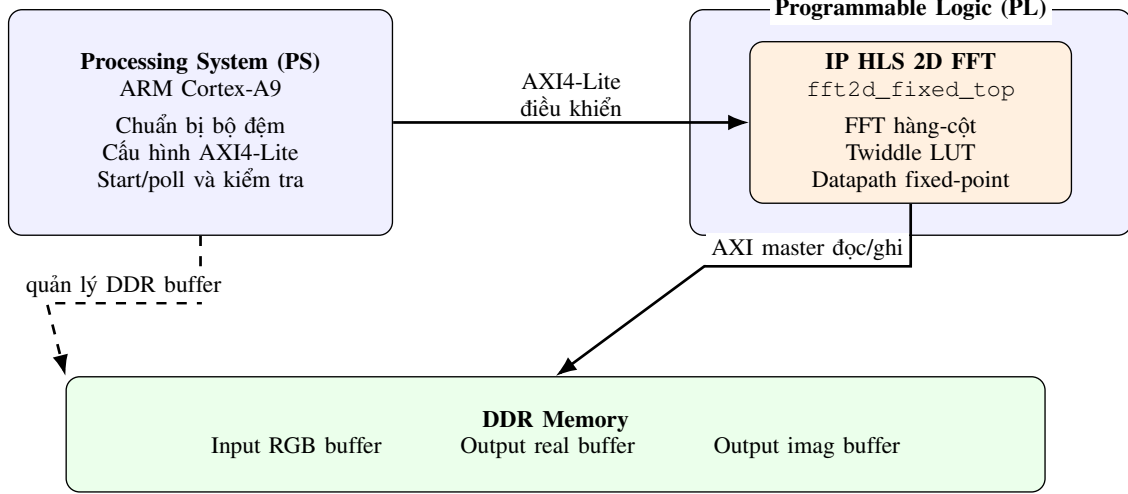


Fig. 1. Kiến trúc tổng quan của bộ tăng tốc 2D FFT trên Zynq.

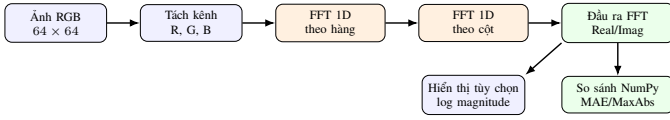


Fig. 2. Luồng xử lý ảnh RGB của bộ tăng tốc đề xuất. Hệ số thực và ảo được dùng cho kiểm chứng định lượng, còn phổ log-magnitude chỉ dùng để quan sát trực quan.



Fig. 3. Datapath nội bộ của bộ tăng tốc 2D FFT hàng-cột. Datapath này được áp dụng độc lập cho các kênh R, G và B.

hợp các phép $\sin$ và $\cos$ tồn tại nguyên trong datapath, nhờ đó giảm mạnh số DSP, LUT và FF.

B. Phân Chia Phần Cứng/Phần Mềm

Bảng I tóm tắt cách phân chia phần cứng/phần mềm. Phía phần mềm xử lý các tác vụ điều khiển và kiểm chứng linh hoạt, còn phần cứng thực hiện phần tính toán FFT lặp lại.

TABLE I
PHÂN CHIA PHẦN CỨNG/PHẦN MỀM

Thành phần	Nhiệm vụ
PS software	Chuẩn bị dữ liệu ảnh, quản lý bộ đệm DDR, cấu hình thành ghi accelerator, flush/invalidate cache, khởi động IP, polling trạng thái hoàn tất, so sánh kết quả với dữ liệu tham chiếu.
PL hardware	Đọc ma trận đầu vào, tính FFT fixed-point theo hàng và cột cho các kênh R/G/B, áp dụng hệ số twiddle, ghi đầu ra phần thực và phần ảo.

C. Tổ Chức Bộ Nhớ Và Giao Tiếp AXI

Kiến trúc baseline sử dụng DDR cho các bộ đệm đầu vào và đầu ra. Bộ đệm đầu vào lưu dữ liệu ảnh RGB, trong khi hai vùng đầu ra lưu hệ số FFT phần thực và phần ảo. Các thanh ghi AXI4-Lite cung cấp thông tin điều khiển và địa chỉ bộ đệm. Các cổng AXI master memory-mapped cho phép accelerator đọc mảng đầu vào và ghi các mảng đầu ra. Cách tổ chức này ưu tiên sự đơn giản trong kiểm chứng; các phiên bản sau có thể bổ sung AXI DMA hoặc AXI Stream để tăng thông lượng.

D. Luồng Kiểm Chứng

Dự án sử dụng luồng kiểm chứng nhiều tầng như Hình 4. Python NumPy được dùng để sinh kết quả FFT tham chiếu. Hiện thực C++ được kiểm tra trước trên desktop. Phiên bản HLS sau đó được kiểm tra qua C simulation, C synthesis và C/RTL co-simulation. Cuối cùng, nền tảng phần cứng sinh ra được dùng bởi ứng dụng bare-metal Vitis để cấu hình và chạy accelerator trên Zynq.

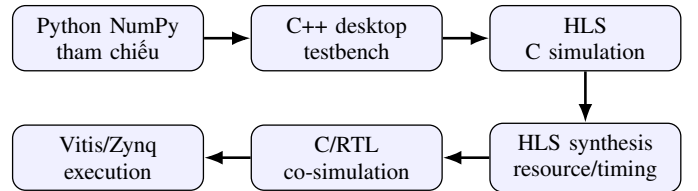


Fig. 4. Luồng kiểm chứng nhiều tầng được sử dụng trong dự án.

V. THIẾT LẬP THỰC NGHIỆM

Thực nghiệm hướng đến bo mạch PYNQ-Z1 dùng chip Zynq-7000 xc7z020clg400-1. Bộ tăng tốc được tổng hợp bằng AMD Vitis HLS 2025.2 với ràng buộc clock 10 ns. Kích thước đầu vào cố định là ảnh RGB 64×64 . Bốn mẫu tổng

hợp gồm square, gradient, checker và circle được dùng để kiểm chứng định lượng. Kết quả Python NumPy fft2 được dùng làm tham chiếu. Sai số được đo trên hệ số FFT thô bằng MAE và MaxAbs cho cả phần thực và phần ảo.

TABLE II
CẤU HÌNH THỰC NGHIỆM

Mục	Cấu hình
Bo mạch	PYNQ-Z1
FPGA part	xc7z020clg400-1
Toolchain	AMD Vitis HLS / Vivado / Vitis 2025.2
Clock mục tiêu	10 ns
Clock ước lượng	7.300 ns
Kích thước đầu vào	RGB 64×64
Kiểu dữ liệu chọn	ap_fixed<28,20>
Mẫu kiểm thử	square, gradient, checker, circle
Tham chiếu	Python NumPy fft2
Thước đo	Latency, MAE, MaxAbs, BRAM, DSP, FF, LUT

VI. KẾT QUẢ VÀ ĐÁNH GIÁ

A. Mức Sử Dụng Tài Nguyên

Tài nguyên phần cứng là thước đo trung tâm của thiết kế vì mục tiêu là triển khai trên FPGA Zynq-7000 có tài nguyên hữu hạn. Bảng III trình bày ước lượng tài nguyên sau HLS synthesis cho cấu hình được chọn.

TABLE III
MỨC SỬ DỤNG TÀI NGUYÊN FPGA

Tài nguyên	Dùng	Tổng	Tỷ lệ
BRAM_18K	66	280	23%
DSP	8	220	3%
FF	9869	106400	9%
LUT	12375	53200	23%

DSP chỉ sử dụng 3%, cho thấy thiết kế còn dư địa để mở rộng song song hoặc bổ sung các khối xử lý ảnh khác. BRAM và LUT là hai nhóm tài nguyên có tỷ lệ sử dụng cao nhất, đạt khoảng 23%, nhưng vẫn nằm trong khả năng của chip mục tiêu.

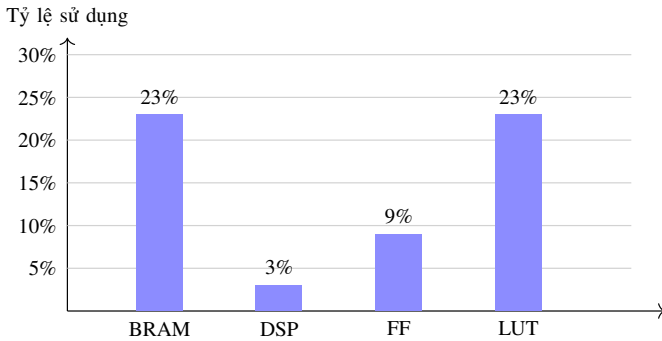


Fig. 5. Biểu đồ tỷ lệ sử dụng tài nguyên của thiết kế trên xc7z020clg400-1.

Ảnh chụp Vivado ở Hình 6 bổ sung góc nhìn sau implementation bên cạnh ước lượng HLS ở Bảng III. Do phạm vi

báo cáo của Vivado là toàn bộ *system wrapper*, các con số không nên được so sánh trực tiếp từng dòng với báo cáo HLS của riêng IP FFT. Tuy vậy, báo cáo này vẫn xác nhận hai điểm quan trọng: thiết kế sau khi tích hợp chỉ sử dụng 8 DSP trên tổng 220 DSP của chip và phần LUT/register vẫn nằm trong giới hạn tài nguyên của xc7z020clg400-1. Điều này giúp củng cố kết luận rằng kiến trúc hiện tại còn đủ dư địa để mở rộng song song hóa hoặc thêm các khối xử lý ảnh phía sau FFT.

B. Ảnh Hưởng Của Tối Ưu Twiddle LUT

Mức giảm tài nguyên lớn nhất đến từ việc thay thế tính $\sin/\cos$ runtime bằng bảng tra twiddle 64 điểm. Bảng IV so sánh phiên bản dùng lượng giác runtime và phiên bản dùng Twiddle LUT.

TABLE IV
SO SÁNH TỐI ƯU TWIDDLE LUT

Phiên bản	BRAM	DSP	FF	LUT
Runtime $\sin/\cos$	42	186	20509	29674
Twiddle LUT	26	8	10313	12126

Số DSP giảm từ 186 xuống 8, tương ứng giảm xấp xỉ 95.7%. FF và LUT cũng giảm hơn một nửa. Kết quả này cho thấy các hàm lượng giác trực tiếp có thể chiếm chi phí lớn trong datapath FFT do HLS sinh ra, và bảng tra là lựa chọn phù hợp cho phép biến đổi có kích thước cố định.

Hình 8 cho thấy bằng chứng trực tiếp từ giao diện báo cáo HLS trong quá trình tối ưu. Ở phiên bản ban đầu, việc để HLS tổng hợp $\sin/\cos$ làm cho phần tính twiddle trở thành một thành phần đắt tài nguyên, đặc biệt ở DSP. Sau khi chuyển sang bảng tra cố định, DSP giảm xuống mức rất thấp trong báo cáo HLS, phù hợp với xu hướng đã lượng hóa trong Bảng IV. Vì hai ảnh là báo cáo theo cây module/loop trong quá trình phát triển, còn Bảng IV là số liệu tổng hợp cuối được chuẩn hóa cho so sánh, bài báo sử dụng bảng làm nguồn số liệu định lượng chính và dùng ảnh chụp như bằng chứng hỗ trợ từ tool.

C. Đánh Giá Bit-Width Fixed-Point

Nhiều cấu hình fixed-point được đánh giá để tìm điểm cân bằng giữa độ chính xác và tài nguyên FPGA. Bảng V tóm tắt quá trình sweep bit-width bằng HLS C simulation trên bốn mẫu RGB. Các cấu hình có số bit phần nguyên không đủ đều không vượt qua ngưỡng sai số hiện tại vì FFT 64×64 có thể tạo ra giá trị trung gian và đầu ra lớn.

Cấu hình ap_fixed<28,20> được chọn vì vượt qua cả bốn mẫu kiểm thử trong khi dùng ít bit hơn ap_fixed<32,20>. Ước lượng tài nguyên cuối của cấu hình này được trình bày riêng ở Bảng III dựa trên HLS synthesis của lần co-simulation được báo cáo. Các cấu hình thất bại cho thấy dải biểu diễn phần nguyên quan trọng hơn việc chỉ tăng độ chính xác phần thập phân đối với kích thước FFT baseline này.

Name	Slice LUTs (53200)	Slice Registers (106400)	F7 Muxes (26600)	Slice (13300)	LUT as Logic (53200)	LUT as Memory (17400)	Block RAM Tile (140)	DSPs (220)	Bonded IOPADs (130)	BUFGCTRL (32)	BSCANE2 (4)
design_1_wrapper	8461	11552	5	3735	7790	671	39.5	8	130	2	1
design_1_i (design_1)	8013	10811	5	3504	7366	647	39.5	8	0	1	0
dbg_hub (dbg_hub)	448	741	0	245	424	24	0	0	0	1	1

Fig. 6. Ảnh chụp báo cáo utilization sau implementation trong Vivado cho thiết kế hệ thống. Báo cáo này phản ánh mức sử dụng tài nguyên ở cấp wrapper, bao gồm IP FFT, kết nối AXI, Processing System, khối reset và logic debug.

TABLE V
SWEEP BIT-WIDTH FIXED-POINT

Định dạng	Pass	Worst Real MaxAbs	Worst Imag MaxAbs	Trạng thái
ap_fixed<32, 20>	4/4	48.1013	34.5814	PASS
ap_fixed<28, 20>	4/4	390.4006	308.7507	PASS
ap_fixed<24, 20>	0/4	4928.0017	3312.7220	FAIL
ap_fixed<22, 20>	0/4	44172.9983	30855.2780	FAIL
ap_fixed<24, 16>	0/4	489472.0039	295416.7260	FAIL
ap_fixed<24, 12>	0/4	520192.0002	326136.7223	FAIL
ap_fixed<20, 12>	0/4	520192.0039	326136.7260	FAIL



Fig. 7. So sánh tài nguyên giữa phiên bản tính lượng giác runtime và phiên bản dùng bảng tra twiddle.

D. Độ Chính Xác Và Độ Trễ

Bảng VI trình bày kết quả C/RTL co-simulation được chạy lại cho cấu hình fixed-point được chọn. Tất cả các mẫu đều vượt qua kiểm chứng so với tham chiếu NumPy. Các giá trị MAE và MaxAbs trong bảng được tổng hợp trên cả ba kênh R, G và B; log chi tiết theo từng kênh được lưu trong `hls/fft2d_accel/results/cosim_28_20_64`.

Độ trễ RTL cố định ở mức 1260411 chu kỳ cho cả bốn mẫu vì kích thước đầu vào và giới hạn vòng lặp là cố định. Ở tần số 100 MHz, độ trễ này tương đương khoảng 12.6 ms cho mỗi frame RGB 64×64 . Các giá trị MAE nhỏ so với dải động của hệ số FFT thô, cho thấy cấu hình fixed-point được chọn vẫn giữ được hành vi số học mong đợi trên tập kiểm thử hiện tại.

E. Timing Và Công Suất Sau Implementation

Ngoài tài nguyên logic, timing closure và công suất là hai yếu tố quan trọng để đánh giá khả năng triển khai thực tế của thiết kế [14], [15]. Hình 9 trình bày ảnh chụp báo cáo timing và power sau implementation. Báo cáo timing cho thấy tất cả ràng buộc thời gian do người dùng chỉ định đều được đáp ứng, với WNS dương 0.668 ns, WHS dương 0.012 ns và

không có endpoint thất bại. Điều này cho thấy thiết kế có thể đóng timing ở cấu hình hiện tại thay vì chỉ dừng ở mức mô phỏng HLS.

Về công suất, báo cáo ước lượng tổng công suất on-chip là 1.493 W, trong đó phần động chiếm 1.355 W và phần tĩnh của thiết bị chiếm 0.138 W. Một điểm đáng chú ý là PS7 chiếm tỷ trọng lớn trong công suất động, trong khi logic, BRAM và DSP của phần PL chỉ chiếm tỷ lệ nhỏ. Kết quả này phù hợp với mục tiêu của thiết kế: IP FFT trong PL tiết kiệm DSP và không tạo ra áp lực công suất lớn so với phần hệ thống xung quanh. Tuy nhiên, do báo cáo power được suy ra từ ràng buộc và activity mặc định, kết quả này nên được xem là ước lượng sau implementation; đo công suất thực tế trên bo mạch sẽ là bước cần bổ sung nếu muốn đánh giá năng lượng chính xác hơn.

F. Kiểm Chứng Trực Quan

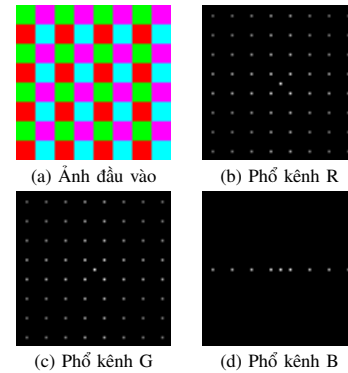


Fig. 10. Ảnh RGB đầu vào và phổ FFT log-magnitude dùng cho kiểm chứng trực quan.

Bên cạnh so sánh định lượng, phổ trực quan giúp xác nhận phép biến đổi tạo ra cấu trúc miền tần số hợp lý. Hình 10 minh họa ảnh checker tổng hợp và phổ log-magnitude tham chiếu Python cho các kênh R, G, B sau khi dịch tâm tần số.

MODULES & LOOPS	ISSUE	VIOLATION TYPE	DISTANCE	LATENCY(NS)	ITERATION LATENCY	INTERVAL	TRIP COUNT	PIPELINED	STRUCTURE	BRAM	DSP	FF	LUT	URAM
✓ ● fft2d_fixed_top (6)							-	no	function	86	197	18,719	25,611	0
> ● fft2d_fixed_top_Pipeline_1 (1)				650.000		64		loop pipeline	function	0	0	8	55	0
> ● fft2d_fixed_top_Pipeline_2 (1)				650.000		64		loop pipeline	function	0	0	8	55	0
> ● fft2d_fixed_top_Pipeline_3 (1)				1.230E5		12,289		loop pipeline	function	0	0	98	283	0
> ● fft2d_fixed_top_Outline_4 (1)	(ii)					-		no	function	0	0	1,410	2,506	0
> ● fft2d_fixed_top_Pipeline_9 (1)				1.230E5		12,289		loop pipeline	function	0	0	133	294	0
> ● Loop 1 (1)	(ii)					-	3	no	loop					

(a) Báo cáo HLS khi hệ số twiddle còn được tính bằng $\sin/\cos$ runtime.

MODULES & LOOPS	ISSUE	VIOLATION TYPE	DISTANCE	LATENCY(NS)	ITERATION LATENCY	INTERVAL	TRIP COUNT	PIPELINED	STRUCTURE	BRAM	DSP	FF	LUT	URAM
✓ ● fft2d_fixed_top (6)							-	no	function	74	8	10,387	12,479	0
> ● fft2d_fixed_top_Pipeline_1 (1)				650.000		64		loop pipeline stpst	function	0	0	8	55	0
> ● fft2d_fixed_top_Pipeline_2 (1)				650.000		64		loop pipeline stpst	function	0	0	8	55	0
> ● fft2d_fixed_top_Pipeline_VITIS_LOOP_ (1)				1.230E5		12,289		loop pipeline stpst	function	0	0	98	283	0
> ● fft2d_fixed_top_Outline_VITIS_LOOP_ (1)	(ii)					-		no	function	0	0	3,501	4,223	0
> ● fft2d_fixed_top_Pipeline_VITIS_LOOP_ (1)				1.230E5		12,289		loop pipeline stpst	function	0	0	133	294	0
> ● VITIS_LOOP_203_8 (1)	(ii)					-	3	no	loop					

(b) Báo cáo HLS sau khi thay bằng bảng tra twiddle cố định.

Fig. 8. Ảnh chụp báo cáo tài nguyên ở mức module/loop của Vitis HLS trong quá trình tối ưu twiddle. Hai ảnh cho thấy xu hướng giảm mạnh DSP và tài nguyên logic khi loại bỏ tính toán lượng giác runtime khỏi datapath FFT.

TABLE VI
ĐỘ CHÍNH XÁC VÀ ĐỘ TRỄ RTL VỚI $ap_fixed<28, 20>$

Mẫu	Latency	Real MAE	Real MaxAbs	Imag MAE	Imag MaxAbs	Trạng thái
square	1260411	1.5991	256.3373	1.1245	91.8750	PASS
gradient	1260411	0.3414	93.2577	0.5035	202.1839	PASS
checker	1260411	1.0716	390.4006	0.5471	308.7507	PASS
circle	1260411	2.9871	240.1185	2.1791	87.1992	PASS

Phần hiển thị trực quan không được dùng làm tiêu chí đúng sai chính vì các bước lấy magnitude, lấy log và chuẩn hóa ảnh có thể che khuất sai số ở mức hệ số. Do đó, kết quả định lượng vẫn dựa trên hệ số thực và ảo của FFT.

VII. THẢO LUẬN

Kết quả thực nghiệm cho thấy bộ tăng tốc HLS đề xuất là một baseline đáng chức năng và tiết kiệm tài nguyên cho xử lý 2D FFT RGB trên Zynq-7000. Tối ưu Twiddle LUT là cải tiến quan trọng nhất, giảm số DSP khoảng 95.7% đồng thời giảm đáng kể FF và LUT. Điều này xác nhận rằng các hàm toán học runtime trong HLS cần được xem xét cẩn thận khi nhắm đến FPGA tài nguyên nhỏ.

Kết quả sweep fixed-point cũng cho thấy vai trò quan trọng của số bit phần nguyên. Các định dạng có ít bit phần nguyên thất bại vì FFT có thể sinh ra giá trị trung gian lớn. $ap_fixed<28, 20>$ là lựa chọn cân bằng trong phạm vi thử nghiệm hiện tại: đủ chính xác trên bốn mẫu kiểm thử và tiết kiệm tài nguyên hơn định dạng rộng hơn.

Báo cáo sau implementation bổ sung thêm bằng chứng về tính khả thi của hệ thống khi tích hợp vào Vivado. Thiết kế đóng timing với slack dương, sử dụng 8 DSP ở cấp hệ thống

và có công suất on-chip ước lượng 1.493 W. Các số liệu này làm rõ rằng kết quả không chỉ là ước lượng thuật toán ở mức C/HLS, mà đã được kiểm tra qua bước triển khai hệ thống. Dù vậy, cần phân biệt phạm vi báo cáo: HLS phản ánh IP FFT, còn Vivado wrapper bao gồm cả PS, interconnect, reset và debug logic.

Thiết kế hiện tại vẫn có một số giới hạn. Hệ thống chỉ hỗ trợ kích thước 64×64 , sử dụng datapath memory-mapped baseline và chưa tối ưu cho streaming video liên tục. Tập kiểm thử đủ cho kiểm chứng ban đầu, nhưng cần thêm nhiều ảnh tự nhiên và benchmark lớn hơn trước khi đưa ra kết luận rộng hơn về chất lượng xử lý ảnh hoặc hiệu năng thời gian thực.

VIII. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Bài báo đã trình bày một bộ tăng tốc 2D FFT cho xử lý ảnh RGB trên FPGA Zynq-7000. Thiết kế sử dụng cấu trúc FFT radix-2 hàng-cột, số cố định và bảng tra twiddle 64 điểm. Bộ tăng tốc được kiểm chứng với kết quả Python NumPy thông qua luồng nhiều tầng gồm C simulation và C/RTL co-simulation. Cấu hình được chọn đạt độ trễ RTL 1260411 chu kỳ cho mỗi frame RGB 64×64 và sử dụng 66 BRAM_18K, 8 DSP, 9869 FF và 12375 LUT trên thiết bị mục tiêu.

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 0.668 ns	Worst Hold Slack (WHS): 0.012 ns	Worst Pulse Width Slack (WPWS): 3.750 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 27708	Total Number of Endpoints: 27692	Total Number of Endpoints: 12659

All user specified timing constraints are met.

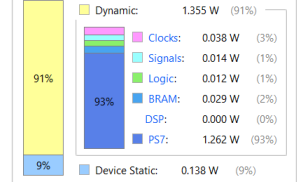
(a) Timing summary sau implementation.

Summary

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power: 1.493 W
Design Power Budget: Not Specified
Process: typical
Power Budget Margin: N/A
Junction Temperature: 42.2°C
 Thermal Margin: 42.8°C (3.6 W)
 Ambient Temperature: 25.0 °C
 Effective θ_{JA} : 11.5°C/W
 Power supplied to off-chip devices: 0 W
 Confidence level: Medium
[Launch Power Constraint Advisor](#) to find and fix invalid switching activity

On-Chip Power



(b) Power summary sau implementation.

Fig. 9. Ảnh chụp báo cáo timing và công suất sau implementation. Timing có slack dương và không có endpoint thất bại; công suất on-chip ước lượng là 1.493 W trong điều kiện activity mặc định/vectorless.

Trong tương lai, hệ thống có thể được mở rộng theo hướng tăng thông lượng và khả năng mở rộng. Các hướng tiếp theo gồm sử dụng AXI DMA hoặc AXI Stream để truyền dữ liệu, pipeline sâu hơn, xử lý song song các kênh RGB, hỗ trợ kích thước ảnh lớn hơn và tích hợp với camera hoặc pipeline video thời gian thực.

LỜI CẢM ƠN

Nhóm tác giả xin cảm ơn giảng viên hướng dẫn và các thành viên phòng thí nghiệm đã hỗ trợ trong quá trình thực hiện dự án.

REFERENCES

- [1] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex fourier series," *Mathematics of Computation*, vol. 19, no. 90, pp. 297–301, 1965.
- [2] Xilinx, *Zynq-7000 SoC Technical Reference Manual*, Xilinx, uG585.
- [3] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. Pearson, 2018.
- [4] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex fourier series," *Mathematics of Computation*, vol. 19, no. 90, pp. 297–301, 1965.
- [5] A. V. Oppenheim and R. W. Schaffer, *Discrete-Time Signal Processing*, 3rd ed. Pearson, 2010.
- [6] K. K. Parhi, *VLSI Digital Signal Processing Systems: Design and Implementation*. Wiley, 1999.
- [7] *Fast Fourier Transform LogiCORE IP Product Guide*, AMD, 2025. [Online]. Available: <https://docs.amd.com/r/en-US/pg109-xfht>
- [8] A. Mukherjee, A. Sinha, and D. Choudhury, "A novel architecture of area efficient fft algorithm for fpga implementation," 2015. [Online]. Available: <https://arxiv.org/abs/1502.07055>
- [9] A. Mukherjee and D. Choudhury, "An area efficient 2d fourier transform architecture for fpga implementation," 2018. [Online]. Available: <https://arxiv.org/abs/1810.06885>
- [10] AMD, *Vitis High-Level Synthesis User Guide*, AMD, uG1399.
- [11] Xilinx, *Zynq-7000 SoC Technical Reference Manual*, Xilinx, uG585.
- [12] R. Rajesh, S. J. Darak, A. Jain, S. Chandhok, and A. Sharma, "Hardware software co-design of statistical and deep learning frameworks for wideband sensing on zynq system on chip," arXiv preprint arXiv:2209.02661, 2022. [Online]. Available: <https://arxiv.org/abs/2209.02661>
- [13] Xilinx, *Vivado Design Suite AXI Reference Guide*, Xilinx, uG1037.
- [14] —, *Vivado Design Suite User Guide: Design Analysis and Closure Techniques*, Xilinx, uG906.
- [15] —, *Vivado Design Suite User Guide: Power Analysis and Optimization*, Xilinx, uG907.